

Mental Health Treatment Prediction Using Distributed Processing and Machine Learning

1. Problem Definition and Motivation

Mental health is one of the most critical global health challenges in modern society. Many individuals experience stress, mood disorders, and behavioral changes without receiving timely treatment. Early identification of individuals who may require mental health support can significantly improve healthcare outcomes and reduce long-term societal and economic costs.

The objective of this project is to design and implement a Big Data system that analyzes a large-scale mental health dataset to predict whether an individual is likely to require treatment. This problem is suitable for a Big Data approach due to the size, variety, and complexity of behavioral and demographic features involved.

Traditional data analysis methods are insufficient to efficiently process and analyze such large datasets. Therefore, distributed computing and scalable machine learning techniques are required to handle the volume and complexity of the data.

2. Architecture Design

The system follows a Big Data ecosystem architecture consisting of seven layers:

Architecture Flow:

Data Source

↓

Data Ingestion

↓

Distributed Storage

↓

Processing Layer (Spark)

↓

Processed Storage

↓

Analytics / Machine Learning

↓

Visualization Layer

Explanation of Layers:

- **Data Source:** Mental health dataset containing demographic, behavioral, and psychological attributes.
- **Data Ingestion:** CSV-based ingestion into the system using Google Colab.
- **Distributed Storage:** Data is stored in a distributed-like format using Parquet files.
- **Processing Layer:** Data processing is performed using Apache Spark for scalable computation.
- **Processed Storage:** Cleaned and transformed data is stored in Parquet format.
- **Analytics / ML:** Machine learning models (Logistic Regression, Random Forest, Gradient Boosted Trees) are applied for classification.
- **Visualization Layer:** Results are visualized using Python libraries and optionally Power BI dashboards.

3. Tool–Layer Justification Table

Layer	Tool Used	Why It Is Appropriate	Alternative Considered
Distributed Storage	Parquet / Simulated HDFS	Efficient columnar storage suitable for large-scale distributed systems	MongoDB (not selected as it is better for operational queries)
Processing	Apache Spark	Supports distributed and scalable data processing	Traditional Pandas (not scalable for large datasets)
Ingestion	CSV / Colab Upload	Simple and effective for batch data ingestion	Kafka (not required for static dataset)
Processed Storage	Parquet Files	Fast read/write and optimized storage format	CSV (less efficient and larger size)
Machine Learning	Spark MLlib	Integrated with Spark and scalable for big data	Scikit-learn (not scalable for large datasets)
Visualization	Power BI	Effective for insight presentation and dashboards	Excel (limited scalability and automation)

4. Implementation

The implementation was carried out using a distributed data processing pipeline:

Data Loading

```
1 df = spark.read.csv("/content/Mental Health Dataset.csv", header=True, inferSchema=True)
```

Feature Engineering

Categorical variables were converted into numerical representations using StringIndexer and OneHotEncoder.

Feature Assembly

```
1 from pyspark.ml.feature import VectorAssembler
2
3 # Correcting the case of the column name in the features list
4 features = [
5     "Gender_index",
6     "Country_index",
7     "Occupation_index",
8     "Growing_Stress_index",
9     "Mood_Swings_index",
10    "Coping_Struggles_index",
11    "Work_Interest_index",
12    "Social_Weakness_index",
13    "Changes_Habits_index",
14    "Days_Indoors_index",
15    "Mental_Health_History_index",
16    "family_history_index"
17 ]
18
19 assembler = VectorAssembler(
20     inputCols=features,
21     outputCol="features_v2"
22 )
23
24 df = assembler.transform(df)
```

Model Training

Three machine learning models were implemented:

- Logistic Regression (baseline model)
- Random Forest Classifier
- Gradient Boosted Trees

Example (Random Forest):

```
1 from pyspark.ml.classification import RandomForestClassifier
2
3 rf = RandomForestClassifier(
4     featuresCol="features_v2",
5     labelCol="treatment_index",
6     numTrees=200,
7     maxBins=64
8 )
9
10 model = rf.fit(train)
```

5. Results and Insights

The models were evaluated using accuracy as the primary metric.

Model	Accuracy
-------	----------

Logistic Regression	~0.714
---------------------	--------

Random Forest	0.687
---------------	-------

Gradient Boosted Trees	0.698
------------------------	-------

Key Insights:

- Logistic Regression performed best, indicating that relationships between features are largely linear.
- Behavioral and psychological factors are strong predictors of treatment need.
- Tree-based models did not significantly outperform linear models due to dataset noise and feature structure.



6. Discussion

Trade-offs:

- Simpler models performed better than complex ensemble models.
- Feature selection significantly impacted performance.

Limitations:

- Dataset contains noisy and subjective variables.
- High-cardinality categorical features may reduce model efficiency.

Challenges:

- Handling categorical variables in Spark required careful encoding.
- Model tuning was limited by computational constraints in Colab.

Future Improvements:

- Incorporate deep learning models.
- Use real-time streaming data ingestion (Kafka).
- Improve feature engineering using domain knowledge.

7. Big Data Justification

This project qualifies as a Big Data solution because:

- It processes a dataset with a large number of records and features(292k).
- It uses distributed processing through Apache Spark.
- It implements a scalable architecture capable of handling large volumes of data.
- Traditional single-machine tools would be inefficient for processing and model training.

Therefore, the system demonstrates core Big Data characteristics: volume, variety, and scalability.

8. Source Code

The full implementation includes:

- Data ingestion scripts
- Feature engineering pipeline
- Machine learning models
- Evaluation scripts
- Visualization code

The code is organized in modular steps within a Colab notebook to ensure reproducibility and clarity.

[Mental Health Dataset](#)

[Colab code](#)